

CORSO DI CYBERSECURITY – EPICODE

REPORT DI LABORATORIO

Gestione dei Permessi di Lettura, Scrittura ed Esecuzione in Linux

Report di Laboratorio

A cura di:

Nicola Madonna

Epicode

27 agosto 2026

Indice

1	Riassunto	2
2	Obiettivi	2
3	Configurazione del Laboratorio	3
3.1	Ambiente di laboratorio	3
3.2	Configurazione di rete	3
3.3	Installazione e configurazione	3
3.4	Verifica	3
4	Analisi dei Dati Raccolti	4
4.1	Script eseguibile (script.sh)	4
4.2	File di configurazione riservato (config.env)	4
4.3	Cartella condivisa (cartella_condivisa)	5
4.4	Riepilogo dei permessi assegnati	5
5	Conclusioni	7

1 Riassunto

Questa esercitazione ha avuto come obiettivo la configurazione e la gestione dei permessi di lettura, scrittura ed esecuzione su file e directory in ambiente Linux, utilizzando una macchina Kali Linux come piattaforma di lavoro. È stata creata una struttura di test composta da uno script eseguibile, un file di configurazione contenente dati sensibili e una cartella pensata per la condivisione tra utenti di uno stesso gruppo. Per ciascun elemento sono stati applicati permessi differenti, coerenti con il ruolo e il livello di riservatezza previsto, tramite il comando `chmod`, verificando ogni modifica con `ls -l`. Il lavoro ha permesso di osservare in pratica come la notazione ottonaria si traduca in permessi effettivi su proprietario, gruppo e altri utenti, e di ragionare sulle motivazioni di sicurezza dietro ogni scelta.

2 Obiettivi

- Comprendere e applicare il modello di permessi Linux (lettura, scrittura, esecuzione) su file e directory.
- Configurare autonomamente una struttura di file/cartelle rappresentativa di scenari reali (script eseguibile, dato riservato, risorsa condivisa).
- Modificare i permessi con `chmod`, sia in notazione simbolica sia numerica.
- Documentare ogni passaggio con screenshot del terminale.
- Motivare, nella relazione, le scelte effettuate riguardo ai permessi assegnati.

3 Configurazione del Laboratorio

3.1 Ambiente di laboratorio

L'esercizio è stato svolto su una macchina Kali Linux, utilizzando l'utente standard `kali` (non root) da terminale (shell `zsh`, di default su Kali). Non sono stati utilizzati strumenti aggiuntivi oltre alle utility di base della shell (`mkdir`, `touch`, `chmod`, `ls`).

3.2 Configurazione di rete

Non applicabile: l'esercizio è stato condotto interamente in locale, sul filesystem della macchina Kali, senza necessità di connettività di rete o comunicazione tra host.

3.3 Installazione e configurazione

È stata creata una cartella di lavoro dedicata, `progetto_permessi`, contenente i tre elementi oggetto dell'esercizio: uno script (`script.sh`), un file di configurazione (`config.env`) e una sottocartella (`cartella_condivisa`).

```
mkdir ~/progetto_permessi && cd ~/progetto_permessi \  
&& touch script.sh config.env \  
&& mkdir cartella_condivisa
```

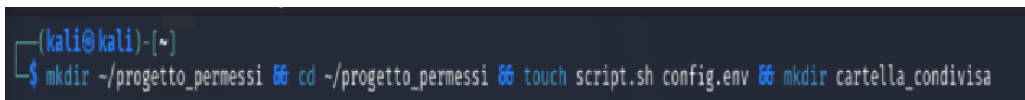


Figura 1: Creazione della struttura di test: cartella di lavoro, script, file di configurazione e sottocartella condivisa.

3.4 Verifica

Subito dopo la creazione, è stato eseguito `ls -l` per osservare i permessi assegnati di default dal sistema (determinati dall'`umask`), da usare come punto di partenza prima delle modifiche.

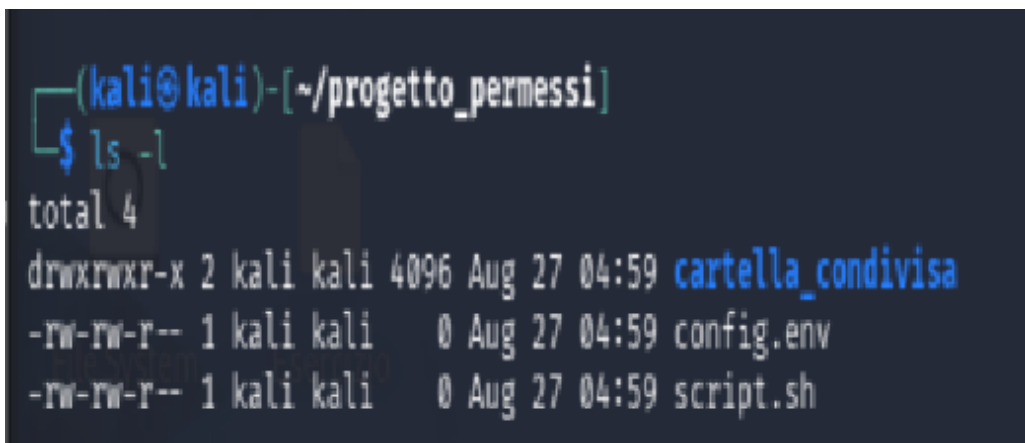


Figura 2: Permessi di default: file a 664 (`rw-rw-r--`) e cartella a 775 (`rw-rwxr-x`).

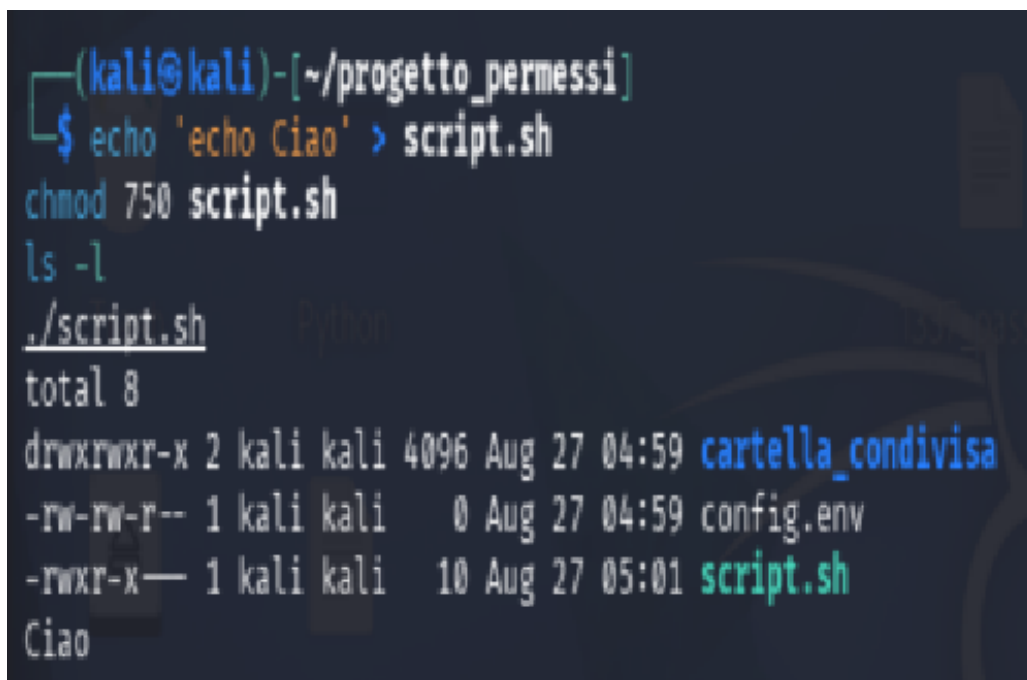
4 Analisi dei Dati Raccolti

Una volta completato il setup, sono stati modificati i permessi dei tre elementi in modo differenziato, in base alla funzione che ciascuno doveva svolgere: esecuzione controllata, riservatezza dei dati e condivisione limitata al gruppo.

4.1 Script eseguibile (script.sh)

Allo script è stato assegnato il permesso 750 (rwxr-x—): il proprietario può leggere, scrivere ed eseguire il file, il gruppo può leggerlo ed eseguirlo ma non modificarlo, mentre agli altri utenti è negato ogni accesso. Questa configurazione riflette un caso tipico di script operativo che deve poter essere lanciato da un gruppo di lavoro fidato, senza però permettere a chiunque di leggerne o alterarne il contenuto. Dopo la modifica, il file è stato eseguito con successo (`./script.sh`), a conferma che il bit di esecuzione era stato applicato correttamente.

```
echo 'echo Ciao' > script.sh
chmod 750 script.sh
ls -l
./script.sh
```



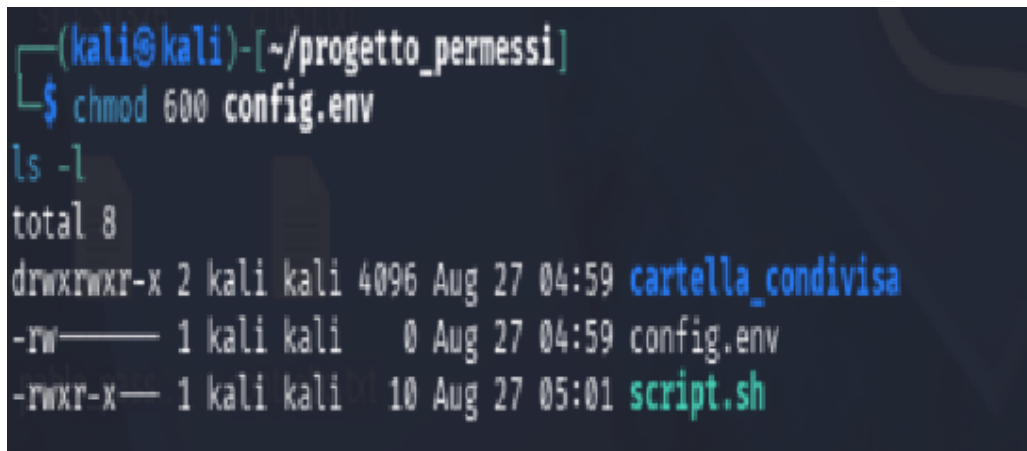
```
(kali@kali)-[~/progetto_permessi]
$ echo 'echo Ciao' > script.sh
chmod 750 script.sh
ls -l
./script.sh
total 8
drwxrwxr-x 2 kali kali 4096 Aug 27 04:59 cartella_condivisa
-rw-rw-r-- 1 kali kali  0 Aug 27 04:59 config.env
-rwxr-x— 1 kali kali  10 Aug 27 05:01 script.sh
Ciao
```

Figura 3: script.sh impostato a 750 e verificato tramite esecuzione diretta.

4.2 File di configurazione riservato (config.env)

A `config.env`, che simula un file contenente dati sensibili (ad esempio credenziali o parametri di configurazione), è stato assegnato il permesso 600 (rw————): solo il proprietario può leggere e scrivere il file, mentre gruppo e altri utenti non hanno alcun accesso. Questa è la configurazione minima consigliata per file che non devono in alcun modo essere consultati da altri utenti del sistema.

```
chmod 600 config.env
ls -l
```



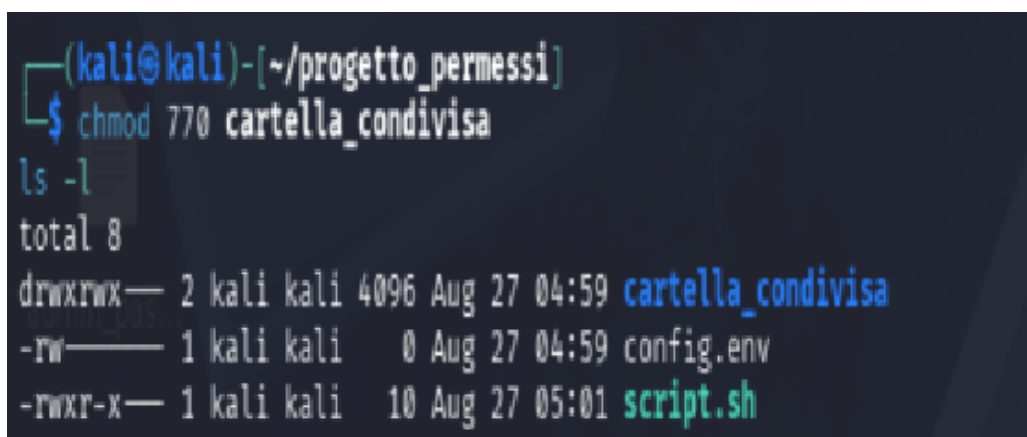
```
(kali@kali)-[~/progetto_permessi]
$ chmod 600 config.env
ls -l
total 8
drwxrwxr-x 2 kali kali 4096 Aug 27 04:59 cartella_condivisa
-rw----- 1 kali kali    0 Aug 27 04:59 config.env
-rwxr-x-- 1 kali kali   10 Aug 27 05:01 script.sh
```

Figura 4: config.env ristretto a 600, accessibile solo dal proprietario.

4.3 Cartella condivisa (cartella_condivisa)

Alla cartella è stato assegnato il permesso 770 (rwxrwx—): proprietario e gruppo hanno pieno accesso (lettura, scrittura ed esecuzione, quest'ultima necessaria per poter attraversare la directory), mentre agli altri utenti è negato qualsiasi accesso. Questa scelta riproduce uno scenario di collaborazione tra membri di uno stesso gruppo di lavoro, che devono poter creare, leggere e modificare file al suo interno, escludendo però utenti esterni al gruppo.

```
chmod 770 cartella_condivisa
ls -l
```



```
(kali@kali)-[~/progetto_permessi]
$ chmod 770 cartella_condivisa
ls -l
total 8
drwxrwx-- 2 kali kali 4096 Aug 27 04:59 cartella_condivisa
-rw----- 1 kali kali    0 Aug 27 04:59 config.env
-rwxr-x-- 1 kali kali   10 Aug 27 05:01 script.sh
```

Figura 5: cartella_condivisa impostata a 770, accessibile solo a proprietario e gruppo.

4.4 Riepilogo dei permessi assegnati

La tabella seguente riassume la configurazione finale ottenuta per i tre elementi.

Tabella 1: Riepilogo dei permessi finali assegnati agli elementi dell'esercizio.

Elemento	Permessi	Notazione	Motivazione
script.sh	750	rwxr-x—	Eseguibile da owner e gruppo, non modificabile da altri
config.env	600	rw——-	Dati riservati, accesso esclusivo al proprietario
cartella_condivisa	770	rw-rwx—	Accesso condiviso al gruppo, escluso il resto del sistema

In tutti e tre i casi il criterio applicato è lo stesso principio di sicurezza del privilegio minimo: ogni elemento riceve solo i permessi strettamente necessari al suo utilizzo previsto, negando per default l'accesso a chiunque non ne abbia bisogno.

5 Conclusioni

L'esercitazione ha permesso di mettere in pratica la gestione dei permessi Linux su tre casi d'uso concreti e distinti, passando dalla teoria della notazione rwx/ottonaria a scelte motivate su file e cartelle reali. È emerso chiaramente come la stessa operazione (`chmod`) possa essere applicata con logiche diverse a seconda della funzione dell'elemento: massima restrizione per i dati sensibili, accesso controllato al gruppo per le risorse condivise, esecuzione selettiva per gli script.

Un aspetto interessante riguarda il ruolo dell'`umask` nella configurazione di default: senza un intervento esplicito, i file sarebbero rimasti leggibili e scrivibili anche dal gruppo (664), un livello di apertura non sempre appropriato, come nel caso di `config.env`. Questo evidenzia l'importanza di non affidarsi ai permessi di default quando si gestiscono dati sensibili.

Come sviluppo futuro, sarebbe utile ripetere l'esercizio con un secondo utente di test per verificare concretamente il diniego di accesso (`Permission denied`), ed eventualmente esplorare l'uso di Access Control List (ACL) tramite `setfacl` per scenari di condivisione più granulari rispetto al modello proprietario/gruppo/altri.